Directional Collapse in Fine-Tuning: Preserving the Way Back

Anonymous ACL submission

Abstract

Fine-tuning a language model to perform a task in one direction can improve that task while weakening its ability to work in the reverse direction. We study this tradeoff across seven task families and test a simple intervention: replace a small share of forward training pairs with their reversals. For example, in format conversion, 1% reversed pairs yield 90.2% backward success, compared with 0.4% under forward-only training, while forward performance is nearly unchanged. The intervention starts from the base model and keeps training-example counts and optimizer steps fixed. Benefits extend to code, translation, and semantic parsing, although the useful proportion and forward costs vary by task. Comparisons with contrastive and other auxiliary objectives show why ordinary reversed-pair training is an essential baseline for evaluating preservation methods.

1 Introduction

A translation pair can train either language direction. A question–SQL pair can train semantic parsing or question generation. Although the same pair supplies both tasks, training on one direction need not preserve the other. Prior work documents directional degradation in code transformation (Nikiema et al., 2025) and translation (Zhu et al., 2024). Evaluating only the trained direction can therefore miss a substantial loss of an existing capability.

The practical intervention is simple: replace some forward examples with their reversed pairs and train with ordinary cross-entropy (CE) loss. In our formatting comparison, forward-only supervised fine-tuning (SFT) improves forward success from 60.2% to 99.7%, but backward success falls from 51.1% to 0.4%. With 1% reversal, backward success is 90.2% and forward success is 99.6%. The mixture is trained from the base checkpoint; it prevents much of the backward loss rather than repairing an already collapsed model with additional data.

We make three connected contributions. First, we characterize forward/backward tradeoffs across task families, models, and training runs. Second, we measure how those tradeoffs change with the proportion of reversed pairs. Third, we test whether specialized objectives add benefits beyond ordinary directional supervision. Across several tasks, small reversal proportions preserve backward performance while retaining most forward gains. This changes how preservation methods should be evaluated: a specialized objective must show what it adds beyond simply training on the reverse mapping.

2 Paired tasks and directional degradation

A paired instance contains two sides (x, y), together with any required context such as a function or database schema. The forward task generates y from x; the backward task generates a valid x from y. Reversing a paired task need not mean uniquely reconstructing a string: translations and questions admit paraphrases, and a function can have several inputs producing the same output. Correctness follows the requested task, rather than reference-string matching whenever a semantic check is available.

We call any reduction in backward success after forward-only training *directional degradation*. For descriptive counting and highlighting, *severe directional collapse* means backward success at most half the base-model rate. This threshold describes the size of a loss; it is not a significance test. All comparisons use the base model evaluated on the same held-out examples as the fine-tuned models.

3 Tasks and experimental setup

What the model must produce. Table 1 defines the seven task families and their correctness checks. Translation contributes four training orientations and algebra two, giving 11 orientations in the main comparison; opposite orientations are not independent datasets. Formatting converts JSON to YAML/XML and Markdown tables to CSV, with the backward tasks converting back. Code pairs readable Python with five obfuscation styles. Execution prediction supplies the function in both directions and accepts any input producing the required backward output. Unit conversion is an additional boundary experiment outside these seven families, testing whether forward training necessarily harms reverse performance (Appendix C.6).

What success means. Each answer receives a binary, task-specific correctness score; strict success is the percentage satisfying that predicate. Format outputs must parse with the target parser, match the underlying data, and avoid copying the input. Copying matters particularly for JSON-to-YAML: JSON also parses as YAML. The copy guard rejects this trivial behavior, although the original scorer does not require syntax exclusive to YAML. Code correctness primarily measures agreement on execution tests, not recovery of original identifier names. Algebra checks symbolic equivalence; factoring also requires a product form, while the expansion scorer has no separate expanded-form requirement.

Translation requires the target language and COMET-22 above a threshold fixed from each base model's score distribution before tuned evaluation. Thresholds differ by model and direction, so success percentages compare training methods within a model, not absolute translation quality across models. The original forward translation check lacks the backward check's explicit copy rejection. SQL generation uses database execution; backward question generation is scored by parsing the question with a frozen Granite model and checking the resulting query. This model-based check can reject valid paraphrases. The other proposed model-based check, WebNLG triples-to-text, has an unreliable reference extractor and contributes no tuned result. Exact prompts, parsing rules, thresholds, source versions and split sizes are consolidated in Appendix A.

Training data and intervention. The retained training sizes are in Table 1. In the reference formatting run, 1% means 65 reversed pairs among 6500; in code it means 62 among 6499. The actual integer counts depend on grouping and filtering; the appendix reports both nominal proportions and realized counts. Forward-only SFT uses all pairs in the forward direction. A replacement mixture partitions pairs by identity, keeping grouped variants of a program or country together, and replaces the selected examples with their reversals. No underlying pair is trained in both directions in a replacement mixture. Every mixture starts from the base model. Training-example counts, epochs, and optimizer steps match forward SFT. This does not guarantee equal token counts, runtime, or floating-point operations (FLOPs); detailed accounting is in Appendix C.2.

Let I_F and I_B be the forward and backward instructions, and partition training pairs D into unreversed F_ρ and reversed B_ρ at replacement proportion ρ. With target-completion loss ℓ_θ, the schematic pair-level objective is

$$\mathcal{L}_\rho = \frac{1}{|D|}\Big[\sum_{(x,y)\in F_\rho} \ell_\theta(y \mid I_F, x) + \sum_{(x,y)\in B_\rho} \ell_\theta(x \mid I_B, y)\Big]. \tag{1}$$

At $\rho = 0$, the model trains only on forward targets. Reversing selected pairs directly supervises the backward conditional using the same underlying relationship.

Models, controls and uncertainty. We evaluate Llama 3B/8B, Gemma 4B/12B, and OLMo 1B, primarily with LoRA. The main reference comparison uses Llama 3B; additional models and training runs appear in Appendix B. Generic instruction replay replaces half of task examples, backward-only training tests backward learnability, and longer-forward and both-direction controls use additional budget. The full dose ladder and controls are in Table 9. A common effective batch and final-checkpoint recipe are detailed in Appendix C.1. Paired cluster-bootstrap intervals measure uncertainty over test examples; ranges across training runs separately show training variability. Neither repeated tasks nor resampled test examples count as independent task families.

Task	Input and required output	Primary success check	Train
Code transformation	Readable Python → obfuscated Python; e.g. renaming `count` to `a0` without changing execution. Backward requests readable code.	Execution agrees on stored tests and output is not a copy. Original identifier names are not required.	6499
Format conversion	JSON → YAML/XML; Markdown table → CSV. Backward converts back; e.g. `{"id": 3}` ↔ `id: 3`.	Target parser accepts output, data structures agree, and output is not a copy. YAML syntax need not be exclusive to YAML.	6500
Translation	English ↔ German/Chinese; both training orientations. E.g. "Good morning" ↔ "Guten Morgen".	Target-language check and COMET-22 above a fixed model/direction threshold; backward additionally rejects copies.	6500
Question → SQL	Question plus schema → SQLite query; backward produces a question. E.g. "How many rows?" ↔ `SELECT COUNT(*) FROM t`.	Forward: equal database results. Backward: a frozen parser maps the question to a query with equal results.	6499
Factual relations	Country → capital, ISO code or TLD; backward identifies the country. E.g. France ↔ Paris.	Normalized first-line exact match, excluding copies. Ambiguous relation values are removed before splitting.	504
Execution prediction	Function and argument → return value; backward finds an argument. For $f(x) = x^2$, $3 \to 9$; backward accepts 3 or -3.	Forward: return value agrees with execution. Backward: any argument executing to the requested value is correct.	302
Algebra	Factored → expanded polynomial, and the opposite training orientation. E.g. $(x+1)(x-1) \leftrightarrow x^2 - 1$.	Symbolic equivalence and no copy; factoring also requires a nonexpanded product. Expansion has no separate form check.	6500

Table 1: Task definitions, illustrative examples, scoring, and retained training pairs (Llama3B). Data and scoring details: Appendix A.

4 Forward gains can hide backward losses

Forward-only SFT substantially reduces backward performance in several tasks, sometimes while improving forward performance. Formatting gives the clearest example: forward success rises from 60.2% to 99.7%, while backward success falls from 51.1% to 0.4%. Code transformation shows a similar tradeoff: forward success rises from 37.5% to 82.4%, but backward success falls from 32.0% to 3.8%. Forward improvement alone would miss both losses.

Across the broader set of 91 evaluated model–task–training-run combinations, 52 meet the severe backward-loss threshold. Table 6 gives model coverage; these are descriptive counts, not pooled significance tests. Variation is substantive: German-to-English training also harms forward translation in the reference run, whereas factual-relation SFT improves both directions. Repeated unit-conversion experiments likewise improve both directions under the original recipe (Appendix C.6). Directional loss is therefore a property to measure, not a necessary outcome of fine-tuning.

5 Small backward doses with ordinary SFT

A small share of reversed pairs can prevent large backward losses, but the useful share depends on the task and direction.

Large gains from small proportions. Formatting backward success is 90.2% with 1% reversal and 96.2% with 5%, versus 0.4% under forward-only SFT. Code backward success is 55.8% at 1% versus 3.8%, while forward success changes from 82.4% to 82.8%. Generic replay and doubled forward training do not yield comparable backward benefits in these comparisons (Table 9). More forward optimization is not a substitute for supplying backward targets.

The 1% benefits recur across 3 formatting runs and 3 code runs with different training seeds. Table 7 reports the range of forward and backward changes for every task with repeated low-dose measurements.

Preservation beyond code and formatting. In English-to-Chinese training, backward translation success is 68.2% with 1% reversal versus 4.0% with forward SFT. Forward success changes from 77.9% to 76.6%, revealing a cost. In question-to-SQL training, backward question success rises from 23.2% to 48.5% at 1% and 55.0% at 5%; for-

Task	Dir.	Base	Forward SFT	1% reversal	5% reversal
German → English	F	73.7	67.1	**67.0**	**67.4**
	B	79.2	**0.0**	**79.4**	**82.0**
English → German	F	79.2	84.8	**84.6**	**85.5**
	B	73.3	64.4	**70.8**	**68.5**
English → Chinese	F	74.1	77.9	**76.6**	**77.4**
	B	75.4	**4.0**	**68.2**	**68.3**
Chinese → English	F	76.0	71.5	**72.5**	**72.1**
	B	74.0	**0.0**	**76.2**	**77.1**
Code transformation	F	37.5	82.4	**82.8**	**84.3**
	B	32.0	**3.8**	**55.8**	**75.2**
Question → SQL	F	49.5	62.4	**62.3**	**60.8**
	B	51.1	**23.2**	**48.5**	**55.0**
Format conversion	F	60.2	99.7	**99.6**	**99.9**
	B	51.1	**0.4**	**90.2**	**96.2**
Factual relations	F	71.1	86.7	**88.1**	**87.4**
	B	65.2	71.1	**73.3**	**75.6**
Polynomial expansion	F	17.3	26.1	**25.5**	**26.8**
	B	14.7	7.8	**14.6**	**20.6**
Polynomial factorization	F	14.6	43.6	**44.0**	**43.8**
	B	17.4	14.7	**10.6**	**15.8**
Execution prediction	F	28.0	29.0	**27.0**	**30.5**
	B	16.0	13.0	**14.0**	**17.5**

Table 2: Forward (F) and backward (B) strict success (%) on Llama 3.2-3B (seed 17). Blue: reversal mixtures; red: SFT backward success at most half the base rate.

ward success at 1% is 62.3% versus 62.4%. Figure 1 displays the joint forward/backward comparison; the gains must be assessed alongside these forward costs.

The useful dose depends on the task. When training polynomial expansion, 1% reversed pairs bring backward factorization success to 14.6%, close to the base rate of 14.7%. The opposite training orientation behaves differently: when training factorization, 1% reversal lowers backward expansion success from 14.7% to 10.6%. At 5% it is 15.8%. Thus a dose that maintains one conditional can harm the other; the mixture must be judged jointly with forward performance and relative to the existing capability. The adverse 1% expansion result occurs in all 3 factorization-training runs. A uniform 1% rule would obscure this finding.

The full ladder evaluates 1, 5, 10, 25, and 50% replacement where available. The complete comparisons and dose criterion are in Appendix B.

6 What specialized objectives add

A preservation method may help because it exposes the model to the reverse mapping. The comparison asks whether its training objective adds a benefit beyond that exposure. We compare auxiliary methods with ordinary reversed-pair SFT and with controls that change exposure without adding the proposed loss. Beating forward-only training is not enough to answer this question. Table 3 reports both task directions.

Methods and controls. Representation-alignment methods encode the two sides independently and train paired representations to be similar, using a symmetric contrastive loss alongside forward-only or mixed-direction CE. Negatives come only from training data. This exposes the paired correspondence without applying backward generation CE. Shuffling the positive pairs tests whether their correspondence matters; repeating ordinary CE on the same examples tests whether extra predictive training can reproduce the benefit. Contrastive fine-tuning (CFT) is a separate, code-specific reconstruction that trains program-equivalence judgments with CE. Other baselines add instruction-conflict unlikelihood or periodically generate a forward answer and train on its reverse mapping with round-trip CE. Exact recipes and realized directional exposure are reported in the appendix.

Generation results. Forward-only representation alignment does not consistently improve backward generation beyond additional-CE or shuffled-pair controls. In code, CFT yields 6.5% backward success versus 75.3% for 5% reversed-pair SFT,

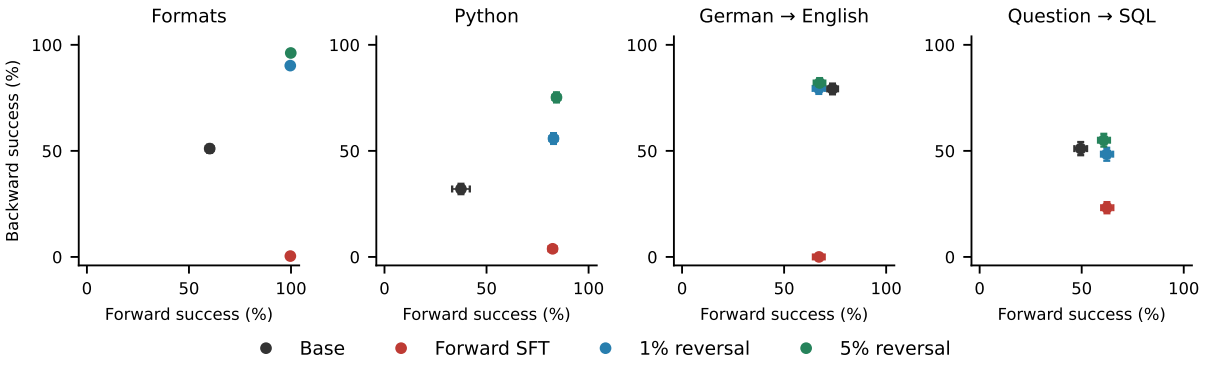


Figure 1: The preservation tradeoff for four tasks, comparing only the base model, forward-only SFT, and 1%/5% reversal. Each panel evaluates Llama3B on the same examples (training seed 17). Bars show marginal 95% cluster-bootstrap intervals over test examples. Table 8 gives paired differences; Table 7 shows variation across training runs.

(a) Contrastive learning and controls

Task	Model	Dir.	Base	SFT	5%	Replay	Rev.	CL-F	CL-M	Shuffle	CE-F+	CE-M+
German → English	G12B	F	76.9	66.9	**66.5**	67.3	1.6	66.1	66.8	66.1	66.5	66.6
		B	72.0	**0.2**	**60.1**	56.2	66.4	0.2	60.9	0.3	0.1	60.4
German → English	G4B	F	75.4	67.7	**67.4**	68.7	40.0	67.2	69.4	67.1	66.9	66.7
		B	75.8	**0.0**	**74.4**	70.4	76.8	0.0	74.5	0.0	0.0	75.1
German → English	L8B	F	74.5	64.2	**65.8**	63.9	4.7	62.8	64.5	63.8	62.8	65.1
		B	73.0	**16.5**	**68.9**	64.3	75.1	0.0	67.7	0.0	18.2	69.6
German → English	L3B	F	73.7	67.3	**67.0**	68.5	63.4	66.5	68.0	66.7	67.0	66.4
		B	79.7	**0.0**	**82.0**	61.0	84.9	0.0	82.9	0.1	0.0	82.2
Format conversion	G4B	F	78.3	100.0	**99.9**	99.8	52.2	100.0	99.9	99.9	99.9	100.0
		B	75.1	**18.7**	**98.0**	49.3	100.0	15.8	99.0	45.0	15.8	99.1
Format conversion	L8B	F	80.2	99.5	**99.9**	99.8	2.2	100.0	99.8	99.7	100.0	99.9
		B	69.0	**0.0**	**98.7**	32.1	99.9	2.2	99.2	0.0	0.0	98.7
Format conversion	L3B	F	60.6	99.7	**99.9**	98.9	27.0	99.6	99.6	99.7	99.2	99.7
		B	51.4	**0.4**	**96.0**	25.6	99.4	0.1	96.0	0.8	0.6	94.2
Unit conversion	G12B	F	33.7	97.1	**97.0**	94.0	44.1	97.6	96.5	97.5	97.4	97.1
		B	35.8	69.8	**79.4**	56.5	94.9	70.3	80.1	74.1	58.7	79.6

(b) Other auxiliary objectives

Task	Model	Dir.	Base	SFT	5%	CFT	Unlikelihood	Round-trip
English → German	L8B	F	71.1	73.0	**73.5**	*n/a*	74.1	72.4
		B	74.2	59.3	**69.9**	*n/a*	0.1	40.6
English → German	L3B	F	79.8	85.5	**85.4**	*n/a*	84.9	84.7
		B	73.3	64.3	**68.5**	*n/a*	0.0	67.1
Code transformation	L3B	F	37.7	82.1	**84.1**	83.9	84.5	81.6
		B	32.8	**3.3**	**75.3**	6.5	0.0	33.4
Question → SQL	L8B	F	62.0	71.5	**71.9**	*n/a*	70.2	71.1
		B	54.9	51.5	**57.8**	*n/a*	6.5	58.6
Question → SQL	L3B	F	49.1	62.0	**60.8**	*n/a*	62.1	61.9
		B	51.4	**24.1**	**55.4**	*n/a*	3.6	48.9

Table 3: Preservation objectives: forward (F) and backward (B) strict success (%), seed 17. CL-F/M: contrastive alignment with forward/mixed CE; CE-F+/M+: additional CE; Shuffle: shuffled pairs. L/G: Llama/Gemma; suffix: billions of parameters. CFT is code-specific (n/a elsewhere). Colors follow Table 2.

with each evaluated on the same examples as its base and controls. Forward-only unlikelihood also fails to preserve backward translation in these experiments. In SQL/Llama3B, backward success is 55.4% with 5% reversal versus 24.1% with forward SFT and 3.6% with unlikelihood. The mixture's forward success is 60.8% versus 62.0% for SFT, showing preservation with a modest forward

cost. These comparisons establish why directional SFT is a necessary baseline.

Mixed-direction alignment sometimes improves on the 5% mixture. In SQL/Llama8B, round-trip training yields 58.6% backward success versus 57.8% for the mixture, while forward success is 71.1% versus 71.9%. These point estimates trade a small forward decrease for a backward increase; their uncertainty is shown in Table 12. Appendix B reports all primary contrasts and their per-run uncertainty. Specialized objectives therefore need to be judged against the directional baseline on both forward and backward performance.

7 Why backward examples may help

Correct-answer likelihood and generated correctness can move in opposite directions. In Gemma translation, forward SFT lowers loss on correct backward answers even as backward generation becomes worse. Correct answers becoming less probable therefore cannot alone account for degradation across models: generation also depends on their probability relative to competing outputs.

We hypothesize that forward-only updates bias which output the model selects for the backward task, and backward examples help maintain that selection. Equation 1 shows how ordinary CE supplies direct supervision for this conditional; it could counter the bias without a special alignment objective. Candidate comparisons support this account: in formatting and translation, forward SFT reduces preference for correct backward answers over copied inputs, while mixed-direction SFT increases that preference.

Layer interventions provide complementary evidence in formatting. Removing middle-layer updates improves backward success beyond scaling and random-removal controls matched for retained update norm, with small forward changes. This suggests selective interference in that task. Gradient and local-update analyses provide no clear collapse-specific signature and are reported with the complete comparisons in Appendix D.

The evidence is exploratory. Restricted candidates do not cover all valid answers, candidate lengths affect rankings, and the layer controls match retained norms rather than edit norms. Multiple interventions in one task do not establish a general mechanism. These observations support output-selection interference as a hypothesis; they do not demonstrate erased knowledge or a general post-training repair.

8 Related work

Directional degradation and reverse training. Code specialization (Nikiema et al., 2025), direction-dependent translation loss (Zhu et al., 2024), and program-execution asymmetry (Hasanov et al., 2026) motivate evaluating existing abilities in both directions. The reversal curse concerns a reverse capability that was never acquired (Berglund et al., 2024). Reverse training addresses direction at the data level (Golovneva et al., 2024); we measure preservation across doses and compare objectives against that supervision. Bidirectional behavior need not imply a unified representation (Coda-Forno et al., 2026).

Forgetting, rehearsal and recovery. Forgetting after fine-tuning (Luo et al., 2023), differences between LoRA and full fine-tuning (Biderman et al., 2024), and rehearsal (Scialom et al., 2022) motivate our controls. Wrapper and offset accounts (Jain et al., 2024; Lee et al., 2024), skewed task inference (Kotha et al., 2024), and update filtering (Abro and Tahir, 2026) concern distinct explanations or later repair. Our replacement intervention preserves during training; behavioral repair alone does not identify a unique internal mechanism.

Objective attribution. Data mixtures can confound attribution (Longpre et al., 2023; Lipton and Steinhardt, 2018). RevThink adds backward questions and reasoning (Chen et al., 2025), illustrating why directional exposure matters even for a bidirectional objective. Reversed-pair SFT, additional CE, and shuffled positives test what an objective adds without assuming auxiliary objectives are universally unnecessary.

9 Conclusion

Training a model in one direction can damage an existing reverse-task ability. In several tested settings, replacing a small fraction of forward examples with reversals preserves much of backward performance while retaining most forward performance. The useful dose and forward cost vary across tasks; some specialized objectives add further benefits. The practical implication is to evaluate both directions and compare new preservation objectives against ordinary directional supervision.

Output-selection diagnostics help interpret this result and remain a supporting hypothesis.

Limitations

Scoring and alternatives. Correctness is task-specific. Code execution on finite tests does not prove semantic equivalence or readable-source recovery. Format equality normalizes scalar case and does not enforce YAML-exclusive syntax. Algebra expansion lacks a separate expanded-form check, and factorization need not be complete. Factual exact matching may reject aliases. Translation thresholds differ by model/direction; its original forward predicate lacks explicit echo rejection. SQL question correctness depends on a frozen parser that can reject valid paraphrases; a keyword guard also rejects questions containing the word "select." The proposed WebNLG text check fails on reference texts and therefore cannot support a tuned comparison (Appendix A).

Scope and generality. Replacement cannot recover information absent from the input. Original code is Python-only; translation covers English paired with German or Chinese. Synthetic variants and changed output contracts reuse semantic corpora rather than supplying independent domains. Some proposed tasks have inadequate starting bidirectional performance or unreliable checks; Appendix C.6 reports these exclusions. Independent OPUS transfer fails its copy-validity criterion, so it establishes no tuned preservation result.

Models, training and resolution. Coverage is uneven across five model sizes and training runs, and most comparisons use LoRA. Full-weight checks cover selected Llama3B runs and do not establish a bound on other recipes. Example and optimizer-step matching does not imply matched tokens or runtime. Paired intervals quantify test uncertainty; training-run ranges do not establish an optimal dose or equivalence between adjacent proportions. Changed prompts and recipes can introduce forward costs or backward loss, including in the original units null case. Historical upstream dataset revisions and training versions are incomplete, limiting exact reconstruction beyond the retained corpora and result analysis.

References

Aarash Abro and Muhammad Tahir. 2026. Spectral unforgetting: Post-hoc recovery of damaged capabilities without retraining. *Preprint*, arXiv:2605.20296.

Lukas Berglund, Meg Tong, Max Kaufmann, Mikita Balesni, Asa Cooper Stickland, Tomasz Korbak, and Owain Evans. 2024. The reversal curse: Llms trained on "a is b" fail to learn "b is a". In *International Conference on Learning Representations*.

Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, et al. 2024. Lora learns less and forgets less. *Transactions on Machine Learning Research*.

Justin Chih-Yao Chen, Zifeng Wang, Hamid Palangi, Rujun Han, Sayna Ebrahimi, Long T. Le, Vincent Perot, Swaroop Mishra, Mohit Bansal, Chen-Yu Lee, and Tomas Pfister. 2025. Reverse thinking makes llms stronger reasoners. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics*.

Julian Coda-Forno, Jane X. Wang, and Arslan Chaudhry. 2026. The illusion of latent generalization: Bi-directionality and the reversal curse. *Preprint*, arXiv:2604.04943.

Olga Golovneva, Zeyuan Allen-Zhu, Jason Weston, and Sainbayar Sukhbaatar. 2024. Reverse training to nurse the reversal curse. *Preprint*, arXiv:2403.13799.

Eshgin Hasanov, Md. Mahadi Hassan Sibat, Shubhra Kanti Karmaker, and Aashish Yadavally. 2026. The path not taken: Duality in reasoning about program execution. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*.

Samyak Jain, Robert Kirk, Ekdeep Singh Lubana, et al. 2024. Mechanistically analyzing the effects of fine-tuning on procedurally defined tasks. In *International Conference on Learning Representations*.

Suhas Kotha, Jacob Mitchell Springer, and Aditi Raghunathan. 2024. Understanding catastrophic forgetting in language models via implicit inference. In *International Conference on Learning Representations*.

Andrew Lee, Xiaoyan Bai, Itamar Pres, et al. 2024. A mechanistic understanding of alignment algorithms: A case study on dpo and toxicity. In *International Conference on Machine Learning*.

Zachary C. Lipton and Jacob Steinhardt. 2018. Troubling trends in machine learning scholarship. *arXiv preprint arXiv:1807.03341*.

Shayne Longpre, Le Hou, Tu Vu, et al. 2023. The flan collection: Designing data and methods for effective instruction tuning. In *International Conference on Machine Learning*.

Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. 2023. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *Preprint*, arXiv:2308.08747.

Serge Lionel Nikiema, Jordan Samhi, Micheline Moumoula, Albarick Djié, Abdoul Kader Kaboré, Jacques Klein, and Tegawendé F. Bissyandé. 2025. Using contrastive learning to improve two-way reasoning in large language models: The obfuscation task as a case study. *Preprint*, arXiv:2509.05553.

Thomas Scialom, Tuhin Chakrabarty, and Smaranda Muresan. 2022. Fine-tuned language models are continual learners. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*.

Dawei Zhu, Pinzhen Chen, Miaoran Zhang, Barry Haddow, Xiaoyu Shen, and Dietrich Klakow. 2024. Fine-tuning large language models to translate: Will a touch of noisy data in misaligned languages suffice? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*.

A Task data, prompts and correctness

Strict success is a binary task-specific predicate, not a common metric across tasks. The primary evaluation uses greedy decoding and the simple instruction without demonstrations. A shared system message and the same backward instruction are used in training and evaluation. Output extraction strips complete or opened Markdown code fences. Echo detection compares Unicode-NFKC/whitespace-normalized strings and also rejects character-trigram Jaccard overlap at least 0.95 when both strings have at least twelve characters. Domain-specific exceptions are stated below; no new scoring rule is applied to the reported results.

Dataset versions and splits. The saved paired JSONL corpora, resolved build configurations, prompt/scorer sources and SHA-256 hashes are listed in the accompanying task-methods metadata. Some original upstream dataset revisions were not recorded; current cached Hub revisions cannot establish the historical training version. The frozen paired corpora therefore specify the reproducible analysis inputs, without implying exact reconstruction of all historical training data.

Shared system message.

> You are a precise transformation tool. You convert the given input into the requested form exactly as instructed, preserving its meaning, and you never change what it says or computes. Reply with the result only: no explanation, no commentary, and no markdown code fences.

A.1 Python transformation

The local Python corpus supplies readable/obfuscated pairs over five transformations: misleading identifier renaming (L1b), random hexadecimal renaming (L1r), sequential minification (L2), control-flow flattening (S1), and opaque predicates/dead code (S2). Training/validation use the transformation-generation pool; held-out tests include only programs available under all five transformations. A program is the uncertainty cluster. Renaming destroys the original names, so the inverse is not unique. Both primary directions require agreement on every stored execution test and reject copies. Transformation-style compliance, original-name recovery, CodeBLEU and readability are not conjuncts of primary success; the workshop readability/similarity predicate is secondary. Built train/validation/test pairs: 6500/500/2060. Retained reference forward-SFT pairs: 6499. Realized reference doses: 1%: 62 reversed pairs among 6499 retained pairs; 5%: 326 reversed pairs among 6499 retained pairs.

Forward prompt.

```
Rewrite the following Python program
so that it is obfuscated in the same
style as the transformation named below,
without changing what it computes.

Transformation: <TRANSFORMATION>

Program:

<INPUT A>
```

Reverse prompt.

```
Recover the original, readable Python
source that this obfuscated program was
produced from.

Program:

<INPUT B>
```

A.2 Serialization conversion

A seeded generator creates lossless JSON/YAML, JSON/XML, and Markdown-table/CSV pairs. JSON uses json.loads, YAML yaml.safe_load, XML an ElementTree decoder, CSV csv.reader, and Markdown a pipe-table parser that removes separator rows. Equality requires matching dictionary keys, container shapes and list order; scalar strings are stripped and lowercased to accommodate XML/CSV type conversion. Copies are rejected in both directions. JSON can parse as YAML: the copy guard excludes unchanged or near-unchanged inputs, but this scorer does not enforce YAML-exclusive syntax or case-sensitive scalar identity. Built train/validation/test pairs: 6500/500/2500. Retained reference forward-SFT

pairs: 6500. Realized reference doses: 1%: 65 reversed pairs among 6500 retained pairs; 5%: 325 reversed pairs among 6500 retained pairs.

Forward / json-yaml prompt.

```
Convert the following JSON document into
YAML, preserving every key and value.
JSON:
<INPUT A>
```

Reverse / json-yaml prompt.

```
Convert the following YAML document into
JSON, preserving every key and value.
YAML:
<INPUT B>
```

Forward / json-xml prompt.

```
Convert the following JSON document into
XML, preserving every key and value.
JSON:
<INPUT A>
```

Reverse / json-xml prompt.

```
Convert the following XML document into
JSON, preserving every key and value.
XML:
<INPUT B>
```

Forward / mdtable-csv prompt.

```
Convert the following Markdown table
document into CSV, preserving every key
and value.
Markdown table:
<INPUT A>
```

Reverse / mdtable-csv prompt.

```
Convert the following CSV document into
Markdown table, preserving every key and
value.
CSV:
<INPUT B>
```

A.3 Machine translation

Training uses the Helsinki-NLP/news_commentary Hub export, German–English and English–Chinese configurations. Both sides must have 20–400 characters; normalized source duplicates are removed. Test data use FLORES-200 devtest from the haoranxu/FLORES-200 mirror. Each orientation trains its designated source-to-target mapping. COMET uses Unbabel/wmt22-comet-da with source, generated translation and reference. Success requires nonempty output, the target-language check, and COMET at or above the frozen threshold. Backward additionally rejects copies; the original forward predicate does not. Chinese requires a Han-character fraction above 0.15; English/German use langdetect probability above 0.5 and reject Han characters. A language-detector exception accepts the language check, a limitation. chrF++ and BLEU are secondary. The threshold is the 25th percentile of scores from 200 untouched-model outputs for each model and direction, before tuned evaluation. Thus success percentages compare training methods within a model/direction, not absolute translation quality across models. Built train/validation/test pairs: 6500/500/1012. Retained reference forward-SFT pairs: 6500. Realized reference doses: 1%: 65 reversed pairs among 6500 retained pairs; 5%: 325 reversed pairs among 6500 retained pairs.

Forward prompt.

```
Translate the following English text
into German.
English:
<INPUT A>
```

Reverse prompt.

```
Translate the following German text into
English.
German:
<INPUT B>
```

A.4 Semantic parsing and verbalization

Questions and queries come from the official database-disjoint xlangai/spider train/validation splits; SQLite databases come from the prem-research/spider mirror, whose pooled question files are not used. The official training split is shuffled into training and validation; official development databases form the test set. Both prompts include a schema summary. Forward outputs must contain the whole word SELECT (case-insensitive) and pass the Spider test-suite execution evaluator on the example database, with plug_value=False and keep_distinct=False; alternatives with equal denotations can pass. Reverse questions are parsed by frozen ibm-granite/granite-3.1-8b-instruct and checked by the same execution comparison. Empty outputs, copied queries and outputs containing that whole word are rejected in reverse. This keyword guard can reject a valid question, as can parser errors. Its

reference-question accuracy is a calibration diagnostic, not a mathematical ceiling for every generated paraphrase. Built train/validation/test pairs: 6499/500/1034. Retained reference forward-SFT pairs: 6499. Realized reference doses: 1%: 65 reversed pairs among 6499 retained pairs; 5%: 325 reversed pairs among 6499 retained pairs.

Forward prompt.

```
Database schema:
<DATABASE SCHEMA>
Write one SQLite query that answers this question.
Question:
<INPUT A>
```

Reverse prompt.

```
Database schema:
<DATABASE SCHEMA>
Write the question, in plain English, that this SQLite query answers.
Query:
<INPUT B>
```

A.5 Country relations

Lots-of-LoRAs/task1146_country_capital, task1314_country_abbreviation and task1320_country_domain_tld provide country–capital, country–ISO-code and country–TLD pairs. Values associated with multiple countries and identical input/target strings are removed; many-to-one currency/calling-code relations are excluded. Splits and uncertainty clusters are countries, keeping their relations together. The scorer uses the first output line, removes a leading "the" or "answer" marker and a trailing period, then compares after Unicode/whitespace normalization. It is case-sensitive and does not accept unrestricted aliases or alternative names. Copying is rejected. Built train/validation/test pairs: 504/69/135. Retained reference forward-SFT pairs: 504. Realized reference doses: 1%: 6 reversed pairs among 504 retained pairs; 5%: 20 reversed pairs among 504 retained pairs.

Forward / capital prompt.

```
Name the capital city of the following. Answer with the name only.
<INPUT A>
```

Reverse / capital prompt.

```
Name the country whose capital is the following. Answer with the name only.
<INPUT B>
```

Forward / abbrev prompt.

```
Name the two-letter ISO country code for the following. Answer with the name only.
<INPUT A>
```

Reverse / abbrev prompt.

```
Name the country whose two-letter ISO code is the following. Answer with the name only.
<INPUT B>
```

Forward / tld prompt.

```
Name the internet top-level domain of the following. Answer with the name only.
<INPUT A>
```

Reverse / tld prompt.

```
Name the country whose internet top-level domain is the following. Answer with the name only.
<INPUT B>
```

A.6 Output and input prediction

The cruxeval-org/cruxeval test export contains 800 Python function/input/output problems. Corpus construction removes identical input/output strings, unexecutable programs, inconsistent reference outputs, and problems where feeding the output back into the function is a valid reverse answer. The retained corpus is split by program. Forward compares the predicted literal with the canonicalized executed return value. Reverse executes the generated argument and accepts any preimage producing the required return value; matching the stored input is unnecessary. Code fences are stripped. Forward answers and reference return values are parsed with ast.literal_eval, falling back to stripped text; generated reverse arguments are passed directly to the execution harness. Execution-based canonicalization determines equality. Built train/validation/test pairs: 302/100/200. Retained reference forward-SFT pairs: 302. Realized reference doses: 1%: 3 reversed pairs among 302 retained pairs; 5%: 15 reversed pairs among 302 retained pairs.

Forward prompt.

```
Given this Python function, what does it return for the input shown?
<FUNCTION CODE>
Call:
f(<INPUT A>)
Write only the returned value, as a Python literal.
```

Reverse prompt.

```
Given this Python function, find an argument that makes it return the value shown.

<FUNCTION CODE>

Required return value:

<INPUT B>

Write only the argument, as a Python literal.
```

A.7 Expansion and factorization

A seeded SymPy generator multiplies primitive integer-coefficient linear factors into degree-2–4 polynomials in one or two variables, using the configured root range. Expanded symbolic expressions are deduplicated across splits. The expansion orientation has factored inputs and expanded targets; the factorization orientation swaps the sides of the same generated pairs. Answers allow caret notation and implicit multiplication. Success requires symbolic equivalence after expansion and excludes copies. Factoring also requires a non-Add top level whose expansion differs from itself; it does not require a canonical or complete irreducible factorization. The expansion scorer has no separate expanded-form conjunct, despite the prompt asking for a polynomial. Built train/validation/test pairs: 6500/500/2500. Retained reference forward-SFT pairs: 6500. Realized reference doses: 1%: 65 reversed pairs among 6500 retained pairs; 5%: 325 reversed pairs among 6500 retained pairs.

Forward prompt.

```
Expand this expression and collect like terms. Write the result as a polynomial.

Expression:

<INPUT A>
```

Reverse prompt.

```
Factor this polynomial completely over the integers. Write it as a product of factors.

Polynomial:

<INPUT B>
```

A.8 Other translation and algebra orientations

The following templates complete the opposite training directions and Chinese pair. Placeholders identify the input side in that orientation; the shared system message is unchanged.

German → English / forward prompt.

```
Translate the following German text into English.

German:

<INPUT A>
```

German → English / reverse prompt.

```
Translate the following English text into German.

English:

<INPUT B>
```

English → Chinese / forward prompt.

```
Translate the following English text into Chinese.

English:

<INPUT A>
```

English → Chinese / reverse prompt.

```
Translate the following Chinese text into English.

Chinese:

<INPUT B>
```

Chinese → English / forward prompt.

```
Translate the following Chinese text into English.

Chinese:

<INPUT A>
```

Chinese → English / reverse prompt.

```
Translate the following English text into Chinese.

English:

<INPUT B>
```

Polynomial factorization / forward prompt.

```
Factor this polynomial completely over the integers. Write it as a product of factors.

Polynomial:

<INPUT A>
```

Polynomial factorization / reverse prompt.

```
Expand this expression and collect like terms. Write the result as a polynomial.

Expression:

<INPUT B>
```

A.9 Translation thresholds

Thresholds below are frozen separately by model, orientation and direction; each row uses the base model calibration described above. They must not be used to rank models by absolute translation quality.

Task orientation	Model	Forward τ	Backward τ
English → German	L3B	0.764888	0.861910
English → German	L8B	0.839859	0.871522
English → German	G4B	0.836251	0.869601
English → German	G12B	0.866418	0.874782
English → German	O1B	0.555442	0.779690
German → English	L3B	0.860885	0.764888
German → English	L8B	0.871522	0.836640
German → English	G4B	0.869446	0.832958
German → English	G12B	0.874782	0.866888
German → English	O1B	0.779690	0.555477
English → Chinese	L3B	0.787774	0.823006
English → Chinese	L8B	0.843215	0.844090
English → Chinese	G4B	0.850115	0.844765
English → Chinese	G12B	0.863640	0.859140
English → Chinese	O1B	0.636863	0.696096
Chinese → English	L3B	0.822380	0.787968
Chinese → English	L8B	0.844090	0.844345
Chinese → English	G4B	0.845060	0.850115
Chinese → English	G12B	0.859140	0.865201
Chinese → English	O1B	0.698151	0.625003

Table 4: Frozen COMET-22 strict-success thresholds.

Model	n	Reference accuracy
G12B	200	63.0%
G4B	200	63.0%
L8B	200	62.5%
L3B	200	64.5%
O1B	200	62.5%

Table 5: Frozen SQL parser calibration on reference questions in the corresponding initial evaluation subset. Variation reflects the subsets and execution layout, not a parser tuned to each evaluated model.

A.10 Model-based correctness checks

SQL-to-question is the model-based check in the main comparison: a frozen Granite parser maps a generated question back to SQL, whose database result is compared with the target. The other proposed round-trip direction is WebNLG triples-to-text: the same model family extracts triples from generated text. Its extractor fails on reference texts, so this task supplies no tuned-model result and is absent from the main task table. Neither check is executable or symbolic verification of the generated natural-language meaning itself.

B Complete model coverage and preservation comparisons

The main comparison displays one reference training run, while the tables below retain variation across runs, paired test uncertainty, the full dose ladder and objective controls. The severe-loss highlighting is descriptive rather than a statistical significance claim. Repeated evaluations use the latest complete result, chosen without inspecting the effect sign; earlier results remain in the accompanying data.

Dose ladder and criterion. The full ladder tests 1, 5, 10, 25, and 50% reversed-pair replacement where available. The prespecified dose knee is the smallest tested proportion reaching 90% of the half-reversal model's backward success. This within-task criterion is neither a common absolute target nor a statistical equivalence test. Small relation/execution corpora yield fewer than one effective batch of distinct reversed examples at low doses; their later exploratory measurements are excluded from the original knee analysis. Replay replaces half of task examples with generic instructions; Rev. trains backward only; Fwd2× doubles forward training; Flip trains both directions at twice the example budget; Multi mixes other tasks. All methods within a task/model/run use the same test examples.

C Additional methods, experiments and boundary conditions

These experiments test forward retention, general ability, sensitivity to prompts and training recipes, and the scope of preservation on other tasks. Exclusions and revised criteria are reported with their scientific reasons. Test outcomes were not used to choose a favorable training run or diagnostic subset. The probe sets contain 541 IFEval

Model	Task orientations	Training runs	Severe losses
gemma3-12b	7	14	8
gemma3-4b	7	21	14
llama31-8b	6	12	7
llama32-3b	11	32	18
olmo2-1b	4	12	5

Table 6: Model–task–training-run coverage. Severe loss: SFT backward success at most half the base rate. Counts are descriptive.

Task	Runs	ΔF_{1-S}	ΔB_{1-S}	ΔF_{5-S}	ΔB_{5-S}
German → English	3	[-1.0, +0.1]	[+76.1, +79.6]	[-0.6, +0.3]	[+80.0, +82.0]
English → German	3	[-0.9, -0.2]	[+6.2, +8.6]	[-1.6, +0.7]	[+4.1, +5.8]
English → Chinese	3	[-1.7, -0.8]	[+49.5, +64.2]	[-1.5, -0.5]	[+49.1, +64.3]
Chinese → English	3	[-0.7, +1.5]	[+76.2, +77.5]	[-0.1, +0.6]	[+77.1, +77.8]
Code transformation	3	[-0.2, +1.7]	[+52.0, +60.0]	[+0.0, +1.9]	[+68.8, +71.4]
Question → SQL	3	[-0.5, +2.0]	[+13.9, +33.4]	[-1.6, -0.5]	[+19.3, +40.0]
Format conversion	3	[-0.5, -0.0]	[+86.1, +89.8]	[-0.2, +0.2]	[+92.2, +95.8]
Polynomial expansion	3	[-1.6, -0.6]	[+6.8, +8.3]	[-2.2, +0.7]	[+12.8, +14.0]
Polynomial factorization	3	[-3.4, +0.4]	[-4.1, -3.4]	[-2.3, +0.1]	[-0.1, +1.1]

Table 7: Ranges across training runs of reversal-mixture minus SFT success (percentage points), Llama3B. F/B: forward/backward; subscripts: reversal proportions. Ranges are not confidence intervals.

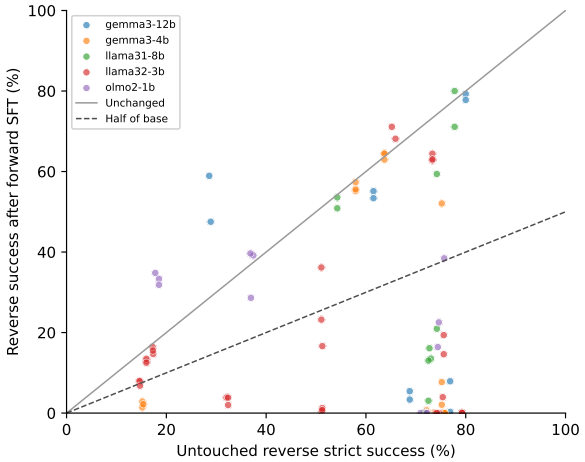


Figure 2: Base and forward-SFT backward success over model–task–training-run combinations. The dashed line marks half the base rate. Repeated tasks and training runs are not independent observations.

prompts and 1,319 GSM8K questions. Independent transfer uses 1,688 OPUS pairs; synthetic magnitude/template tests use 1,000 examples each. Recipe comparisons cover 48 separately trained variants.

C.1 Paired data and reference training recipe

Each task supplies fixed training, validation, and test pairs. Content and pair-identity overlap checks exclude held-out test material during corpus construction; validation is not used to select a preferred test result. Opposite orientations of a task share the underlying task family. The reference recipe and actual corpus/trainer counts are reported in Table 14; smaller factual-relation and execution corpora are retained at their actual size. Seed-specific partitions define reversed-pair replacement. Training summaries retain realized supervised-token exposure and length filtering, rather than equating pair count with tokens. Every reported tuned comparison includes a freshly evaluated base and all declared arms on the same retained instance set.

Recorded reference recipe. The main-table Llama3B experiments use the common training recipe: LoRA rank 32, scaling 64, dropout 0.05, learning rate 0.0001, 3 epochs, effective batch 64, and maximum sequence length 2048 tokens. The scheduler is cosine with warm-up fraction 0.03, weight decay 0 and gradient clipping 1. Training uses bfloat16 and final checkpoints without intermediate checkpoint selection. LoRA targets query, key, value and output attention projections and the gate, up and down MLP projections. Allocation-specific microbatches and accumulation retain the recorded effective batch. Full-weight and robustness variants are separate recipes, not substitutions for this reference.

Task	Mixture	Forward change	Backward change
Format conversion	1%	-0.0 [-0.3, +0.2]	+89.8 [+88.6, +90.9]
Format conversion	5%	+0.2 [-0.0, +0.4]	+95.8 [+95.0, +96.5]
Code transformation	1%	+0.4 [-0.9, +1.7]	+52.0 [+49.4, +54.5]
Code transformation	5%	+1.9 [+0.5, +3.3]	+71.4 [+68.9, +73.8]
German → English	1%	-0.1 [-1.6, +1.5]	+79.4 [+76.9, +81.9]
German → English	5%	+0.3 [-1.4, +2.0]	+82.0 [+79.6, +84.3]
Question → SQL	1%	-0.1 [-1.5, +1.3]	+25.2 [+22.1, +28.3]
Question → SQL	5%	-1.5 [-3.2, +0.0]	+31.8 [+28.5, +35.0]

Table 8: Reversal-mixture minus SFT success (percentage points), with 95% paired cluster-bootstrap intervals (2,000 draws; unadjusted).

		Reference		Reversed-pair share					Other adaptations				
Task	Dir.	Base	SFT	1%	5%	10%	25%	50%	Replay	Rev.	Fwd2×	Flip	Multi
German → English	F	73.7	67.1	**67.0**	**67.4**	66.2	68.6	67.4	68.6	63.5	57.8	64.4	69.1
	B	79.2	**0.0**	**79.4**	**82.0**	82.8	82.3	82.9	59.0	85.2	1.4	83.9	71.5
English → German	F	79.2	84.8	**84.6**	**85.5**	85.4	84.7	84.6	85.4	0.0	77.8	84.8	84.0
	B	73.3	64.4	**70.8**	**68.5**	68.2	68.2	67.8	72.0	66.2	64.4	67.0	73.0
English → Chinese	F	74.1	77.9	**76.6**	**77.4**	77.1	78.6	76.7	78.6	0.0	75.9	80.4	77.0
	B	75.4	**4.0**	**68.2**	**68.3**	69.9	71.5	71.9	70.9	71.2	9.5	72.8	70.3
Chinese → English	F	76.0	71.5	**72.5**	**72.1**	72.3	72.2	71.8	74.8	23.4	63.8	71.5	73.3
	B	74.0	**0.0**	**76.2**	**77.1**	80.0	78.3	77.2	1.0	77.7	0.0	76.1	71.0
Code transformation	F	37.5	82.4	**82.8**	**84.3**	83.6	84.4	83.9	74.8	65.6	88.3	86.3	70.5
	B	32.0	**3.8**	**55.8**	**75.2**	76.5	80.3	85.6	14.5	86.3	9.6	86.6	3.4
Question → SQL	F	49.5	62.4	**62.3**	**60.8**	62.0	60.1	58.6	59.8	42.0	61.1	60.8	56.8
	B	51.1	**23.2**	**48.5**	**55.0**	57.5	59.9	58.1	17.1	59.1	27.3	58.8	50.4
Format conversion	F	60.2	99.7	**99.6**	**99.9**	99.8	99.8	99.8	99.0	25.2	99.0	99.8	97.8
	B	51.1	**0.4**	**90.2**	**96.2**	97.0	98.6	99.6	24.6	99.4	0.7	99.6	42.9
Factual relations	F	71.1	86.7	**88.1**	**87.4**	88.1	86.7	87.4	88.1	80.0	88.9	89.6	83.0
	B	65.2	71.1	**73.3**	**75.6**	75.6	80.0	81.5	67.4	83.7	58.5	81.5	65.9
Polynomial expansion	F	17.3	26.1	**25.5**	**26.8**	27.3	25.5	24.8	25.8	14.6	29.6	27.7	21.2
	B	14.7	7.8	**14.6**	**20.6**	25.3	31.0	36.5	9.2	43.6	6.7	44.9	6.0
Polynomial factorization	F	14.6	43.6	**44.0**	**43.8**	43.2	40.2	36.1	36.9	7.8	49.6	44.7	26.8
	B	17.4	14.7	**10.6**	**15.8**	18.0	23.0	24.9	14.0	26.2	12.7	27.4	26.5
Execution prediction	F	28.0	29.0	**27.0**	**30.5**	28.5	29.0	30.0	27.5	21.0	32.0	29.0	24.5
	B	16.0	13.0	**14.0**	**17.5**	20.5	26.5	28.5	13.0	29.0	12.5	28.5	12.5

Table 9: Full reversal-dose ladder and controls: forward (F) and backward (B) strict success (%), Llama 3.2-3B, seed 17. Control definitions appear in Appendix B.

C.2 Dose, forward retention, and computational cost

The main figure plots forward and backward strict success for the base model, forward SFT, and the two smallest reversed-pair doses. The full ladder and secondary controls are reported separately. Panels retain model, task orientation, and seed identity. Arms lacking a common evaluation campaign are not connected as paired observations. The units boundary case is shown alongside collapse cells. Repeated passes use the latest complete evaluation, with earlier passes retained in the audit; selection never uses effect sign.

Paired contrasts use the domain's instance/cluster unit. Training-seed variation is separate: resampling test instances does not measure training variability. No aggregate pools cells as independent domains. The full dose ladder and knee criterion are in Appendix B; overlapping intervals do not establish equivalence.

The cost table reports available trainer runtime, optimizer steps, and CE-corpus tokens. Auxiliary encoded-side exposure is retained separately in the analysis. Missing timing, GPU-type, queue-time, or exposure records remain unavailable. Additional CE is the declared proxy; corpus token counts alone do not establish matched objective FLOPs or wall time.

Scoped finding. On the representative translation campaign, reversed-pair SFT preserves reverse behavior with nearly unchanged forward task performance relative to forward SFT (Ta-

Task	Model	Seed	Δ CL-fwd versus SFT	Δ CL-mix5 versus mix5
fmt	gemma3-4b	17	-2.88 [-3.88, -1.96]	+1.08 [+0.32, +1.88]
fmt	gemma3-4b	42	-1.28 [-1.96, -0.64]	-1.04 [-1.80, -0.32]
fmt	gemma3-4b	1234	+2.36 [+1.60, +3.20]	-0.32 [-0.92, +0.24]
fmt	llama31-8b	17	+2.24 [+1.52, +3.04]	+0.48 [-0.12, +1.12]
fmt	llama31-8b	42	-33.12 [-35.48, -30.64]	+0.40 [-0.20, +1.00]
fmt	llama31-8b	1234	+0.00 [+0.00, +0.00]	-0.72 [-1.52, +0.12]
fmt	llama32-3b	17	-0.24 [-0.56, +0.04]	+0.00 [-1.12, +1.12]
fmt	llama32-3b	42	-1.48 [-2.12, -0.92]	+1.68 [+0.52, +2.88]
fmt	llama32-3b	1234	-0.52 [-1.00, -0.04]	+4.44 [+3.08, +5.84]
mt_de-en	gemma3-12b	17	+0.00 [+0.00, +0.00]	+0.79 [-1.78, +3.56]
mt_de-en	gemma3-12b	42	-0.10 [-0.40, +0.00]	+0.59 [-2.08, +3.26]
mt_de-en	gemma3-12b	1234	+0.00 [+0.00, +0.00]	+0.69 [-1.68, +3.16]
mt_de-en	gemma3-4b	17	+0.00 [+0.00, +0.00]	+0.10 [-2.27, +2.27]
mt_de-en	gemma3-4b	42	+0.00 [+0.00, +0.00]	+0.40 [-2.08, +2.77]
mt_de-en	gemma3-4b	1234	+0.00 [+0.00, +0.00]	-0.30 [-2.47, +2.17]
mt_de-en	llama31-8b	17	-16.50 [-19.47, -13.64]	-1.19 [-4.05, +1.88]
mt_de-en	llama31-8b	42	-13.54 [-16.50, -11.07]	-0.40 [-3.26, +2.57]
mt_de-en	llama31-8b	1234	-23.32 [-26.78, -19.96]	-1.28 [-4.35, +1.58]
mt_de-en	llama32-3b	17	+0.00 [+0.00, +0.00]	+0.89 [-1.38, +3.26]
mt_de-en	llama32-3b	42	+0.00 [+0.00, +0.00]	+2.47 [+0.00, +5.04]
mt_de-en	llama32-3b	1234	-0.10 [-0.40, +0.00]	+0.10 [-2.57, +2.87]
units	gemma3-12b	17	+0.50 [-1.70, +2.80]	+0.70 [-0.90, +2.30]
units	gemma3-12b	42	-1.40 [-2.70, -0.20]	+0.40 [-1.10, +2.00]
units	gemma3-12b	1234	-4.90 [-7.00, -3.10]	+0.50 [-0.90, +1.90]

Table 10: Contrastive learning minus directional SFT: backward-success changes (percentage points), with 99% paired cluster-bootstrap intervals. Five primary contrasts are adjusted within each task/model/run.

ble 15). Generic replay recovers less reverse performance in that campaign. The units case improves both ways under ordinary SFT; robustness results show that preservation and forward costs depend on task and recipe. Recorded budgets support within-recipe comparisons, not matched wall-time claims across different objectives.

C.3 General ability and overlap controls

The fixed panel is German-to-English/Llama3B, format conversion/Gemma4B, and units/Gemma12B. Each campaign compares base, SFT, replay, mixed-direction SFT, forward-only and mixed-direction contrastive arms, and their additional-CE controls. The auxiliary arm is fixed rather than selected by its best test score.

IFEval measures instruction following without demonstrations; GSM8K measures numerical reasoning using fixed training demonstrations. A neutral assistant system prompt is shared by all systems. Dataset revisions and overlap exclusions are recorded before generation. Auditing covers both sides of every local training/validation corpus, replay, and mixed-task sources, using normalized exact/contained strings. Fixed demonstrations are checked against the retained test set. Lexical auditing does not establish semantic or pretraining independence; shared numerical structure limits interpretation of the units/GSM8K comparison.

All systems were evaluated on the same examples; generated-output differences from the base were checked to detect an adapter that failed to load. Strict IFEval and strict GSM8K endpoints are reported separately; loose instruction checks and flexible numeric extraction remain secondary. Endpoint changes are paired within each campaign and seed variation is retained. A concentration claim is scoped to this declared panel; unrelated benchmark scales are not subtracted as if interchangeable.

Scoped finding. General retention depends on the endpoint and task (Table 17). Formatting/Gemma4B loses numerical reasoning under forward SFT while instruction-following changes are small; replay recovers more of that reasoning endpoint than mixed-direction SFT. Translation/Llama3B loses instruction following while improving numerical reasoning. Reverse-task recovery therefore does not establish general-ability preservation, and the evidence does not support a universal disproportionate-loss claim across benchmarks.

C.4 Independent real-data transfer

Existing German-to-English checkpoints are evaluated on the independently sourced OPUS-100

Task	Model	Seed	CL-fwd vs extra CE	CL-fwd vs shuffled	CL-mix5 vs extra CE
fmt	Gemma 4B	17	+0.04 [-0.84, +0.96]	-29.20 [-31.80, -26.72]	-0.08 [-0.72, +0.56]
fmt	Gemma 4B	42	-16.04 [-18.00, -14.20]	-19.44 [-21.52, -17.48]	-0.44 [-1.28, +0.44]
fmt	Gemma 4B	1234	+8.20 [+6.80, +9.64]	-22.16 [-25.08, -19.24]	-0.60 [-1.24, +0.00]
fmt	llama31-8b	17	+2.24 [+1.52, +3.04]	+2.24 [+1.52, +3.04]	+0.44 [-0.16, +1.00]
fmt	llama31-8b	42	-21.52 [-23.64, -19.44]	-33.00 [-35.40, -30.52]	+0.16 [-0.32, +0.68]
fmt	llama31-8b	1234	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]	-0.48 [-1.24, +0.28]
fmt	Llama 3B	17	-0.48 [-0.96, -0.08]	-0.68 [-1.16, -0.24]	+1.76 [+0.56, +2.96]
fmt	Llama 3B	42	-1.76 [-2.48, -1.12]	-3.40 [-4.32, -2.52]	+0.64 [-0.44, +1.68]
fmt	Llama 3B	1234	-0.36 [-0.88, +0.12]	+0.28 [+0.08, +0.60]	+2.76 [+1.48, +4.00]
mt_de-en	gemma3-12b	17	+0.10 [+0.00, +0.40]	-0.10 [-0.40, +0.00]	+0.49 [-2.17, +2.96]
mt_de-en	gemma3-12b	42	+0.00 [+0.00, +0.00]	-0.10 [-0.40, +0.00]	-1.19 [-3.56, +1.19]
mt_de-en	gemma3-12b	1234	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]	-0.40 [-2.87, +2.08]
mt_de-en	Gemma 4B	17	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]	-0.59 [-2.77, +1.58]
mt_de-en	Gemma 4B	42	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]	+0.00 [-2.27, +2.27]
mt_de-en	Gemma 4B	1234	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]	+1.09 [-1.09, +3.36]
mt_de-en	llama31-8b	17	-18.18 [-21.34, -15.12]	+0.00 [+0.00, +0.00]	-1.88 [-5.04, +1.28]
mt_de-en	llama31-8b	42	-14.53 [-17.49, -11.96]	-0.20 [-0.59, +0.00]	-0.40 [-3.26, +2.47]
mt_de-en	llama31-8b	1234	-21.74 [-25.10, -18.38]	-0.10 [-0.49, +0.00]	-1.09 [-4.15, +1.68]
mt_de-en	Llama 3B	17	+0.00 [+0.00, +0.00]	-0.10 [-0.40, +0.00]	+0.69 [-1.68, +3.16]
mt_de-en	Llama 3B	42	+0.00 [+0.00, +0.00]	-0.10 [-0.40, +0.00]	+1.78 [-0.59, +4.25]
mt_de-en	Llama 3B	1234	-0.10 [-0.40, +0.00]	-0.10 [-0.40, +0.00]	-0.69 [-3.26, +1.88]
units	gemma3-12b	17	+11.60 [+8.50, +14.90]	-3.80 [-5.60, -2.00]	+0.50 [-1.00, +2.00]
units	gemma3-12b	42	-1.10 [-2.50, +0.20]	-0.10 [-1.80, +1.60]	+1.00 [-0.50, +2.60]
units	gemma3-12b	1234	-2.60 [-4.60, -0.80]	-4.80 [-7.00, -2.70]	+0.80 [-0.60, +2.30]

Table 11: Contrastive learning versus additional-CE and shuffled-pair controls: backward-success changes (percentage points), with 99% paired cluster-bootstrap intervals adjusted within each task/model/run.

German–English test split.[1] The panel includes Llama3B and Gemma4B across the fixed training runs and the same SFT, replay, mixed-direction, contrastive, and additional-CE systems. Dataset revision, character-length inclusion rules, directions, overlap exclusions, and scorer are frozen before generations. A new prompt on an existing semantic corpus remains a diagnostic rather than independent transfer evidence.

Overlap auditing includes all local training, validation, replay, and original test sources. Gold, echo, empty, and invalid-output oracles run on compute. Original model-specific COMET thresholds are retained. Strict success, continuous COMET, chrF++, echo, and off-target rates remain separate; checkpoint choice, decoding, and scoring are not tuned on this corpus.

The base model is evaluated first. A task comparison requires initial success of at least 10% in each direction, format failures at most 15%, and copied-input strict success at most 2%. These requirements separate an existing bidirectional ability from a scoring failure or a capability that the model did not possess.

Completed measurements. On every transfer subset, copying the input exceeds the permitted strict-success rate. This violates the scorer-validity requirement and excludes tuned-model comparisons. The base-only boundary trials are retained in Table 18; no tuned checkpoint comparison is eligible. These measurements therefore do not establish preservation on this independent corpus, and the failed criterion is not evidence of adapter collapse.

Versioned diagnostics. Revised output contracts were specified after inspecting unsuccessful task/scorer checks. The original exclusions remain reported separately. Separate versions test bounded signed C++ literal syntax, explicit serialization/conversion/logic contracts, and a WebNLG extractor supplied with a global training-only predicate schema. Translation transfer additionally excludes source copies in both directions, retaining the original thresholds. Reference-answer, copied-input and invalid-answer tests check the revised scorers before comparing tuned models; the base model must have adequate performance in both directions. Revised criteria and prompts define new cells; their results do not replace original failures or support cross-version paired contrasts. Failed oracles and genuine capability limits remain reported boundaries.

[1] https://huggingface.co/datasets/Helsinki-NLP/opus-100.

Task/model	Objective	Compared with	Forward change	Backward change
English → German/L8B	Round-trip	Forward SFT	-0.6 [-2.6, +1.2]	-18.7 [-21.7, -15.9]
English → German/L8B	Round-trip	5% reversal	-1.1 [-3.4, +1.0]	-29.2 [-32.5, -26.1]
English → German/L8B	Unlikelihood	Forward SFT	+1.1 [-0.9, +3.0]	-59.2 [-62.1, -56.2]
English → German/L8B	Unlikelihood	5% reversal	+0.6 [-1.5, +2.6]	-69.8 [-72.4, -67.0]
English → German/L3B	Round-trip	Forward SFT	-0.8 [-2.8, +1.1]	+2.8 [+0.4, +5.1]
English → German/L3B	Round-trip	5% reversal	-0.7 [-2.7, +1.4]	-1.4 [-4.1, +1.0]
English → German/L3B	Unlikelihood	Forward SFT	-0.6 [-2.2, +1.1]	-64.3 [-67.1, -61.3]
English → German/L3B	Unlikelihood	5% reversal	-0.5 [-2.5, +1.5]	-68.5 [-71.3, -65.7]
Code transformation/L3B	CFT	Forward SFT	+1.8 [+0.2, +3.3]	+3.1 [+1.9, +4.2]
Code transformation/L3B	CFT	5% reversal	-0.1 [-1.8, +1.5]	-68.9 [-71.4, -66.4]
Code transformation/L3B	Round-trip	Forward SFT	-0.6 [-2.1, +0.8]	+30.1 [+28.4, +31.7]
Code transformation/L3B	Round-trip	5% reversal	-2.5 [-4.0, -1.1]	-41.9 [-44.3, -39.5]
Code transformation/L3B	Unlikelihood	Forward SFT	+2.3 [+1.0, +3.6]	-3.3 [-4.1, -2.6]
Code transformation/L3B	Unlikelihood	5% reversal	+0.4 [-1.1, +1.8]	-75.3 [-77.7, -73.0]
Question → SQL/L8B	Round-trip	Forward SFT	-0.4 [-1.9, +1.1]	+7.2 [+4.5, +9.8]
Question → SQL/L8B	Round-trip	5% reversal	-0.8 [-2.5, +1.0]	+0.8 [-1.3, +2.9]
Question → SQL/L8B	Unlikelihood	Forward SFT	-1.3 [-3.3, +0.7]	-45.0 [-48.2, -42.0]
Question → SQL/L8B	Unlikelihood	5% reversal	-1.6 [-3.6, +0.3]	-51.4 [-54.4, -48.3]
Question → SQL/L3B	Round-trip	Forward SFT	-0.1 [-1.6, +1.4]	+24.9 [+21.7, +28.3]
Question → SQL/L3B	Round-trip	5% reversal	+1.1 [-0.6, +2.7]	-6.5 [-8.9, -4.1]
Question → SQL/L3B	Unlikelihood	Forward SFT	+0.1 [-1.4, +1.5]	-20.5 [-23.2, -17.7]
Question → SQL/L3B	Unlikelihood	5% reversal	+1.3 [-0.3, +2.9]	-51.8 [-54.9, -48.7]

Table 12: Objective differences in strict success (percentage points), with 95% paired bootstrap intervals. Positive values favor the first method; intervals are unadjusted.

Evaluation	Model/task combinations	Training seeds	Runs
General probes	3	17,42,1234	9
Prompt/template tests	4	17,42,1234	12
Independent transfer	2	17,42,1234	6
Recipe evaluations	2	17,42,1234	6

Table 13: Additional evaluation coverage by task/model, training seeds, and runs.

Completed versioned-domain pilots. Table 19 reports every completed pilot under the revised contracts, including all training runs and both directions. Formatting/Llama8B and units/Gemma12B improve backward performance under forward-only SFT; their mixed-direction gains are additional backward learning rather than restoration of a collapsed capability. Typed Python/C++ at Gemma12B recovers backward success with mixed-direction SFT, with seed-dependent forward costs. At Llama8B, the mixture fails to recover backward behavior consistently, whereas generic replay retains it much better; backward-only training also fails. These results bound the remedy and separate backward learnability from directional preservation. This panel tests only the half-reversal mixture, so it supplies no evidence for a small-dose optimum. Boolean tasks still lack adequate initial bidirectional performance. The WebNLG extractor fails its reference-text oracle, so no D2T tuned comparison is eligible.

C.5 Prompt and training-recipe robustness

The prompt panel includes translation/Llama3B, format conversion/Gemma4B, units/Gemma12B, and Python–C++/Llama8B. Frozen variants retain the original primary instruction and add fixed task/contract prefixes. Both directions use the same semantic instances, decoding limits, and scorers. Every frozen prompt is retained, separating echo/off-target output, semantic error, and measurement failure. Best-prompt summaries remain exploratory recovery diagnostics.

Synthetic extrapolation additionally uses exact unit magnitudes outside original support and behavior-disjoint piecewise-quadratic Python/C++ functions. Original allowlisted scorers and input ranges remain unchanged; reference outputs are checked on all generated instances. These tests do not establish unrestricted natural-code, loop/list, or independent-domain generality.

The recipe panel fixes lower/higher learning-rate and LoRA-rank variants around the original recipe, varying the factors separately. Param-

Task orientation	Train	Val.	Test	Retained	Eval.	Steps	Batch × accum.
German → English	6500	500	1012	6500	1012	306	16 × 4
English → German	6500	500	1012	6500	1012	306	4 × 16
English → Chinese	6500	500	1012	6500	1012	306	4 × 16
Chinese → English	6500	500	1012	6500	1012	306	4 × 16
Code transformation	6500	500	2060	6499	2060	306	4 × 16
Question → SQL	6499	500	1034	6499	1034	306	16 × 4
Format conversion	6500	500	2500	6500	2500	306	16 × 4
Factual relations	504	69	135	504	135	24	16 × 4
Polynomial expansion	6500	500	2500	6500	2500	306	4 × 16
Polynomial factorization	6500	500	2500	6500	2500	306	4 × 16
Execution prediction	302	100	200	302	200	15	16 × 4

Table 14: Paired-corpus sizes and recorded reference training counts. Retained rows, optimizer steps, and batch sizes describe the actual SFT runs.

Task	Model	Seed	Arm	Forward (%)	Reverse (%)
mt_de-en	llama32-3b	17	base	73.7	79.2
mt_de-en	llama32-3b	17	sft	67.1	0.0
mt_de-en	llama32-3b	17	replay	68.6	59.0
mt_de-en	llama32-3b	17	mix5	67.4	82.0
mt_de-en	llama32-3b	17	mix50	67.4	82.9
units	gemma3-12b	17	base	33.7	35.8
units	gemma3-12b	17	sft	97.4	70.0
units	gemma3-12b	17	replay	94.0	56.6
units	gemma3-12b	17	mix50	94.9	90.9

Table 15: Representative preservation results: forward and backward strict success (%), seed 17.

eter values, cells, seeds, and primary contrasts are fixed in the recorded protocol. The comparisons preserve alpha/rank ratio, effective batch, paired-data identity, direction mixture, sequence limit, epoch count, and optimizer schedule. The audit records realized exposure, step count, and available GPU cost. SFT and mixed-direction SFT are compared on format conversion/Llama3B and units/Gemma12B, retaining collapse and null cases.

New adapters were stored separately. The original arms, base, and all recipe variants were regenerated together in each campaign. Tables retain every adverse/null outcome, paired intervals and training-seed ranges; no best-recipe selection supports the primary claim.

Completed measurements. The frozen prompt panel retains directional loss under forward SFT for translation and format conversion; unit conversion remains a null-collapse case under the original recipe (Table 20). Preservation is not uniform across tasks or recipes: the Python–C++ mixture has large seed-dependent forward costs, and the higher learning-rate unit recipe includes reverse loss as well as improvement (Table 21). The unit boundary case is therefore scoped to the original recipe. Every frozen prompt and recipe is retained; selecting the best variant does not support the primary claim. These seed ranges are descriptive, and endpoint scales are not pooled into an overall preservation score.

C.6 Initial task feasibility and full-weight checks

Proposed task comparisons require the base model to perform both directions and require correctness checks that accept reference answers while rejecting copies and invalid outputs. Table 22 states which requirements fail for the original small-model extensions. Excluding these comparisons avoids interpreting an initial floor or a scorer failure as fine-tuning damage. Original unit-conversion experiments at Gemma12B improve both directions under forward SFT; these repeated null results show that degradation is not inherent to every paired task. Python/C++ translation at Llama8B provides an additional successful starting-task comparison. The revised-contract results above reuse their underlying corpora; they are not new domains.

Full-weight fine-tuning compares selected Llama3B runs with forward-only and 5
training (Table 25). Each full checkpoint uses an isolated inference engine with identical held-out

Task	Model	Arm	Minutes	Steps	CE corpus tokens
mt_de-en	llama32-3b	sft	8.9	306	1096262
mt_de-en	llama32-3b	mix5	8.9	306	1096262
mt_de-en	llama32-3b	mix50	9.0	306	1096262
mt_de-en	llama32-3b	replay	8.9	306	1096110
mt_de-en	llama32-3b	sft	8.9	306	1096262
mt_de-en	llama32-3b	mix5	8.9	306	1096262
mt_de-en	llama32-3b	replay	8.9	306	1096110
mt_de-en	llama32-3b	cl_fwd	241.9	306	1096262
mt_de-en	llama32-3b	cl_mix5	62.2	306	1096262
mt_de-en	llama32-3b	sft_extra_ce	123.2	306	1096262
mt_de-en	llama32-3b	mix5_extra_ce	119.1	306	1096262
fmt	gemma3-4b	sft	30.6	306	1927990
fmt	gemma3-4b	mix5	30.7	306	1927882
fmt	gemma3-4b	replay	30.2	306	1927978
fmt	gemma3-4b	cl_fwd	194.3	306	1927990
fmt	gemma3-4b	cl_mix5	193.5	306	1927882
fmt	gemma3-4b	sft_extra_ce	191.3	306	1927990
fmt	gemma3-4b	mix5_extra_ce	194.0	306	1927882
units	gemma3-12b	sft	21.6	306	788819
units	gemma3-12b	mix50	22.1	306	788819
units	gemma3-12b	replay	21.5	306	799547
units	gemma3-12b	sft	21.6	306	788819
units	gemma3-12b	mix5	15.6	306	788819
units	gemma3-12b	replay	21.5	306	799547
units	gemma3-12b	cl_fwd	82.8	306	788819
units	gemma3-12b	cl_mix5	83.0	306	788819
units	gemma3-12b	sft_extra_ce	166.9	306	788819
units	gemma3-12b	mix5_extra_ce	168.0	306	788819

Table 16: Training runtime (minutes), optimizer steps, and CE-corpus sequence tokens before epoch repetition (seed 17). Token counts exclude auxiliary encoded-side exposure.

Cell	Endpoint	Seeds	SFT–base	mix5–SFT	Replay–SFT
fmt/Gemma4B	gsm8k	3	[-56.6, -36.3]	[+0.5, +21.8]	[+24.7, +46.0]
fmt/Gemma4B	ifeval	3	[-1.3, +0.6]	[-3.0, +0.9]	[-8.3, -4.1]
mt_de-en/Llama3B	gsm8k	3	[+6.6, +8.7]	[-2.7, +1.4]	[-13.7, +10.6]
mt_de-en/Llama3B	ifeval	3	[-10.9, -9.6]	[-0.4, +4.4]	[+0.2, +3.0]
units/Gemma12B	gsm8k	3	[+1.2, +2.0]	[-0.7, +0.9]	[-10.4, -9.1]
units/Gemma12B	ifeval	3	[-3.0, +1.5]	[+0.0, +3.5]	[-13.5, -3.0]

Table 17: General-ability changes in strict success (percentage points): minimum/maximum across training runs. Ranges are not confidence intervals.

instances, prompts, tokenizer and scoring settings. Small changes must be interpreted against the measured decoding floor across engine restarts. The separate saved-checkpoint spectral repair experiment concerns later repair, whereas replacement mixtures preserve during training. Its improvement in both directions is a scoped result and does not establish a general internal cause.

C.7 Result selection and reproducibility

All completed comparisons and null/adverse outcomes are retained. Repeated evaluations use the latest complete evaluation, independently of effect sign. Revised scorers and prompts are separate exploratory measurements, with their original failures preserved; differences across versions are not paired treatment effects. Test-pair bootstrap uncertainty is reported separately from variation across training seeds. An interval spanning zero does not show equivalence, and a degenerate interval at a generation floor needs particular caution.

The accompanying data record fixed corpora, prompts, scorer sources, training configurations, per-instance generations and analysis commands. Hashes and complete paired coverage are checked before table generation; failed adapter loading excludes an evaluation. Some historical training code and upstream dataset revisions were not retained, so reproducible result analysis does not imply exact historical training reconstruction. These missing versions are explicit limitations, rather than inferred from a current cached dataset.

Model	Seed	Direction	Base (%)	Format failure (%)	Echo (%)	Campaign
gemma3-4b	1234	forward	33.9	10.1	6.6	blocked
gemma3-4b	1234	reverse	44.9	6.3	0.0	blocked
gemma3-4b	17	forward	33.9	10.0	6.6	blocked
gemma3-4b	17	reverse	44.8	6.3	0.0	blocked
gemma3-4b	42	forward	33.9	10.1	6.6	blocked
gemma3-4b	42	reverse	44.8	6.3	0.0	blocked
llama32-3b	1234	forward	33.7	10.6	6.8	blocked
llama32-3b	1234	reverse	53.0	6.8	0.0	blocked
llama32-3b	42	forward	33.7	10.6	6.8	blocked
llama32-3b	42	reverse	52.9	6.8	0.0	blocked
llama32-3b	17	forward	33.7	10.6	6.8	blocked
llama32-3b	17	reverse	52.8	6.8	0.0	blocked

Table 18: OPUS transfer: base-model capability and scoring-validity checks under the original thresholds. Failed checks exclude tuned comparisons.

C.8 Cross-domain paired-judgment baseline

Code-equivalence CFT is specific to programs. We separately define a paired-judgment extension for English–German translation and question–SQL pairs, comparing Llama3B and Llama8B at training seed 17.

PJ-5 selects the same pairs as the 5% reversal mixture and replaces their generation targets with Yes/No judgments. The remaining 95% retain ordinary forward targets. The model receives both sides and judges whether the German sentence translates the English source, or whether the SQL query answers the question in the supplied schema. Judgment classes differ by at most one example. There are 325 judgments among 6,500 translation rows and 325 among 6,499 SQL rows. No backward generation target is used. Training-example counts and optimizer steps match SFT and reversal; token counts and runtime may differ and are recorded separately.

Positive judgments use original paired examples. Negative candidates come only from the training split and exclude duplicate sources and every observed correct target for that source. Translation uses mismatched training sentences; occasional unrecorded semantic matches remain a label-noise limitation. SQL negatives stay within the same database: both queries must execute, and the frozen execution-equivalence check must reject the candidate. A scorer error cannot supply a negative label. Negative labels are allocated deterministically among the selected pairs that admit such candidates. A shuffled-label control uses identical inputs and class counts with randomized Yes/No labels. All methods are evaluated together on the original held-out generation tasks with their unchanged correctness checks. Paired 95% cluster-bootstrap intervals compare both directions against SFT, 5% reversal, and the shuffled control.

D Likelihood, candidate selection and layer interventions

Five experiments study candidate likelihood, local gradients and layer-specific interference. The subset was chosen before these diagnostic outcomes to cover distinct questions within the available computational budget. These single-seed analyses are exploratory; paired intervals measure test-pair uncertainty, not training-seed stability. All candidates, controls, bands and local steps remain in the evidence bundle. Completed diagnostic experiments: 5. In the completed Llama formatting and translation diagnostics, forward SFT lowers gold completion loss in the forward direction and raises it in reverse; mixed-direction SFT lowers reverse loss relative to SFT with small mean forward changes (Table 26). The local SFT gradient cosines are near zero (Table 27); these checkpoints provide little support for a simple persistent gradient-conflict account. Gradients at these final checkpoints do not reconstruct interference along the training trajectory. This descriptive evidence concerns answer likelihood and does not establish stored knowledge. The Gemma translation diagnostic differs: reverse gold completion loss improves relative to base despite collapsed free generation. Absolute answer loss therefore does not uniformly track collapse. In the three collapse cells, per-token gold-over-copy preference falls under forward SFT and recovers under mixed-direction SFT; the units null cell retains strong preference (Table 28). Total and per-token rankings are reported together because candidate

Versioned task	Model	Seed	Dir.	Base	SFT	50%	Replay	Rev.
Formatting contract	L8B	17	F	34.4	99.9	99.9	99.9	0.0
			B	60.0	63.0	100.0	54.8	99.9
Formatting contract	L8B	42	F	34.9	99.9	99.9	99.7	0.1
			B	60.9	61.6	100.0	16.5	99.9
Formatting contract	L8B	1234	F	34.4	99.9	100.0	99.7	1.1
			B	60.0	66.6	100.0	15.7	99.9
Units contract	G12B	17	F	39.9	97.0	95.1	92.2	38.7
			B	46.9	70.3	89.9	65.0	95.7
Units contract	G12B	42	F	39.9	97.3	93.8	92.9	27.2
			B	46.9	73.3	88.0	73.0	95.6
Units contract	G12B	1234	F	39.9	97.2	94.4	94.4	9.1
			B	46.9	71.2	88.6	70.4	95.9
Typed Python/C++	G12B	17	F	86.2	100.0	100.0	9.2	0.0
			B	95.0	**35.0**	100.0	0.0	100.0
Typed Python/C++	G12B	42	F	86.2	100.0	85.0	36.1	0.0
			B	95.0	72.7	100.0	30.5	100.0
Typed Python/C++	G12B	1234	F	86.2	74.8	73.5	2.3	0.0
			B	95.0	55.8	100.0	0.0	100.0
Typed Python/C++	L8B	17	F	84.2	100.0	100.0	99.6	0.0
			B	100.0	**0.0**	48.4	99.4	0.0
Typed Python/C++	L8B	42	F	85.9	100.0	1.8	99.9	0.0
			B	100.0	**0.0**	0.0	99.4	0.0
Typed Python/C++	L8B	1234	F	85.9	100.0	100.0	95.9	0.0
			B	100.0	**0.0**	0.1	97.8	0.0

Table 19: Revised-output task results: forward (F) and backward (B) strict success (%). Reversal uses 50% replacement. Red: SFT backward success at most half the base rate; underlined: SFT gains in both directions.

lengths differ. Translation has multiple valid answers; a restricted candidate ranking is neither correctness nor a measure of retained knowledge. At the SFT checkpoint, both tested reverse and mixed local-step sizes lower reverse held-out loss in every cell, including the null cell; this is not a collapse-specific signature or a demonstrated generation repair. All checkpoint states, update directions, step sizes and both endpoints are retained in the bundle. Formatting middle-band removals improve reverse success beyond all matched controls with small forward changes, whereas the earliest band harms forward performance without consistent reverse recovery (Table 29). This supports depth selectivity in one cell, with unadjusted intervals across multiple interventions, rather than a universal causal mechanism.

Cell	Setting	Seeds	ΔF_{S-B}	ΔR_{S-B}	ΔF_{M-S}	ΔR_{M-S}
fmt/G4B	original/contract_first	3	[+20.0, +20.2]	[-58.5, -27.8]	[+0.0, +0.1]	[+51.9, +82.1]
fmt/G4B	original/primary	3	[+21.4, +21.7]	[-60.0, -27.9]	[+0.0, +0.2]	[+51.8, +83.8]
fmt/G4B	original/reword	3	[+22.2, +22.2]	[-59.8, -27.8]	[-0.0, +0.1]	[+51.8, +83.5]
mt_de-en/L3B	original/contract_first	3	[-4.4, -4.2]	[-79.0, -78.7]	[-0.5, +0.6]	[+79.9, +81.1]
mt_de-en/L3B	original/primary	3	[-7.7, -5.5]	[-79.7, -79.6]	[-0.3, +0.7]	[+79.2, +82.0]
mt_de-en/L3B	original/reword	3	[-4.6, -3.1]	[-79.1, -78.9]	[-0.7, +0.5]	[+78.8, +81.6]
py_cpp/L8B	original/contract_first	3	[+4.4, +4.4]	[-100.0, -100.0]	[-92.5, +0.0]	[+0.0, +50.0]
py_cpp/L8B	original/primary	3	[+15.8, +15.8]	[-100.0, -100.0]	[-97.8, +0.0]	[+0.0, +48.7]
py_cpp/L8B	original/reword	3	[+6.4, +6.4]	[-100.0, -100.0]	[-89.2, +0.0]	[+0.0, +49.9]
py_cpp/L8B	py_cpp_piecewise_quadratic/primary	3	[-68.2, -68.2]	[+0.0, +0.0]	[+0.0, +0.0]	[+99.3, +100.0]
units/G12B	original/contract_first	3	[+63.6, +63.6]	[+32.5, +36.6]	[-2.3, -1.4]	[+17.1, +20.7]
units/G12B	original/primary	3	[+63.3, +63.3]	[+34.0, +38.0]	[-2.1, -1.4]	[+17.4, +21.3]
units/G12B	original/reword	3	[+65.3, +65.6]	[+47.3, +50.7]	[-2.3, -1.6]	[+17.4, +20.6]
units/G12B	units_magnitude/primary	3	[+35.1, +47.2]	[+36.7, +40.0]	[-1.4, +5.0]	[+6.0, +8.5]

Table 20: Prompt/template sensitivity: ranges of strict-success changes across training runs (percentage points). S/B/M: forward SFT/base/reversal mixture; F/R: forward/backward. Mixtures use 5% for translation/format and 50% for units/Python–C++. Ranges are not confidence intervals.

Cell	Setting	Seeds	ΔF_{S-B}	ΔR_{S-B}	ΔF_{M-S}	ΔR_{M-S}
fmt/L3B	lr_high	3	[+39.5, +39.5]	[-51.1, -49.2]	[-0.2, -0.0]	[+95.4, +98.2]
fmt/L3B	lr_low	3	[+38.7, +39.1]	[-43.0, -40.6]	[+0.2, +0.6]	[+82.6, +86.6]
fmt/L3B	original	3	[+39.4, +39.5]	[-50.8, -49.8]	[-0.1, +0.2]	[+92.2, +95.7]
fmt/L3B	rank_high	3	[+39.0, +39.3]	[-51.1, -51.0]	[-0.0, +0.4]	[+95.8, +96.6]
fmt/L3B	rank_low	3	[+38.2, +39.4]	[-50.4, -45.4]	[-0.0, +1.1]	[+90.8, +92.9]
units/G12B	lr_high	3	[+63.6, +63.7]	[-28.2, +12.1]	[-4.2, -1.1]	[+43.7, +84.6]
units/G12B	lr_low	3	[+62.8, +63.1]	[+36.1, +37.4]	[-3.4, -2.5]	[+14.1, +16.2]
units/G12B	original	3	[+63.3, +63.6]	[+34.1, +37.8]	[-2.3, -1.2]	[+17.7, +21.2]
units/G12B	rank_high	3	[+63.4, +64.3]	[+13.7, +27.9]	[-2.7, -1.9]	[+27.7, +41.1]
units/G12B	rank_low	3	[+62.8, +63.6]	[+25.6, +37.6]	[-3.1, -0.4]	[+15.9, +28.3]

Table 21: Training-recipe sensitivity: ranges of strict-success changes across training runs (percentage points). S/B/M: forward SFT/base/reversal mixture; F/R: forward/backward. Ranges are not confidence intervals.

Task	Model	Reason an original comparison is excluded
units	L3B	forward success below 10%; forward format failures above 15%; backward success below 10%; backward format failures above 15%
units	G4B	forward format failures above 15%; backward success below 10%; backward format failures above 15%
logic	L3B	forward success below 10%; forward format failures above 15%; backward success below 10%; backward format failures above 15%
logic	G4B	forward success below 10%; forward format failures above 15%; backward success below 10%
py_cpp	L3B	forward format failures above 15%; backward format failures above 15%
py_cpp	G4B	forward success below 10%; forward format failures above 15%

Table 22: Reasons for excluding the initial task extensions.

Task	Model	Forward	Reverse	Eligibility
units	llama32-3b	0.4	1.9	Fail
units	gemma3-4b	37.6	3.5	Fail
logic	llama32-3b	0.0	0.0	Fail
logic	gemma3-4b	0.0	0.0	Fail
py_cpp	llama32-3b	78.6	74.1	Fail
py_cpp	gemma3-4b	5.1	99.9	Fail

Table 23: Initial task eligibility: base-model forward and backward strict success (%).

Task	Model	Seed	Arm	Forward	Reverse
py_cpp	llama31-8b	17	base	85.9	100.0
py_cpp	llama31-8b	17	sft	100.0	0.0
py_cpp	llama31-8b	17	mix50	100.0	48.7
py_cpp	llama31-8b	17	replay	100.0	97.6
py_cpp	llama31-8b	17	rev	0.0	0.0
py_cpp	llama31-8b	42	base	84.2	100.0
py_cpp	llama31-8b	42	sft	100.0	0.0
py_cpp	llama31-8b	42	mix50	1.7	0.0
py_cpp	llama31-8b	42	replay	99.9	99.5
py_cpp	llama31-8b	42	rev	0.0	0.0
units	gemma3-12b	1234	base	33.7	35.8
units	gemma3-12b	1234	sft	96.9	72.7
units	gemma3-12b	1234	mix50	95.1	90.7
units	gemma3-12b	1234	replay	94.8	71.9
units	gemma3-12b	1234	rev	47.3	94.7
units	gemma3-12b	17	base	33.7	35.8
units	gemma3-12b	17	sft	97.4	70.0
units	gemma3-12b	17	mix50	94.9	90.9
units	gemma3-12b	17	replay	94.0	56.6
units	gemma3-12b	17	rev	44.0	95.0
units	gemma3-12b	42	base	33.7	35.8
units	gemma3-12b	42	sft	97.3	73.7
units	gemma3-12b	42	mix50	95.7	91.3
units	gemma3-12b	42	replay	95.5	72.7
units	gemma3-12b	42	rev	45.9	94.6
py_cpp	llama31-8b	1234	base	84.2	100.0
py_cpp	llama31-8b	1234	sft	100.0	0.0
py_cpp	llama31-8b	1234	mix50	100.0	0.2
py_cpp	llama31-8b	1234	replay	95.8	97.9
py_cpp	llama31-8b	1234	rev	0.0	0.0

Table 24: Larger-model comparisons: forward and backward strict success (%), with methods evaluated on the same examples.

Task	Seed	Direction	Base	Full SFT	Full mix5
code	17	forward	37.9	70.0	70.7
code	17	reverse	31.9	2.0	65.9
mt_de-en	17	forward	73.3	70.9	72.0
mt_de-en	17	reverse	80.1	72.8	83.6
mt_en-de	17	forward	80.1	87.0	86.6
mt_en-de	17	reverse	72.9	69.9	72.4
mt_en-zh	17	forward	73.9	79.1	79.2
mt_en-zh	17	reverse	75.9	66.9	71.8
mt_zh-en	17	forward	76.3	74.2	73.9
mt_zh-en	17	reverse	73.8	70.2	78.5
sql	17	forward	49.3	60.5	59.7
sql	17	reverse	50.4	18.7	55.1

Table 25: Full-weight fine-tuning: forward and backward success (%), Llama3B. Comparisons span engine restarts.

Task	Model	Direction	SFT minus base	mix5 minus SFT
fmt	llama32-3b	forward	-0.117 [-0.132, -0.101]	+0.000 [-0.000, +0.001]
fmt	llama32-3b	reverse	+0.121 [+0.104, +0.139]	-0.288 [-0.331, -0.247]
mt_de-en	llama32-3b	forward	-0.277 [-0.336, -0.220]	+0.004 [-0.003, +0.012]
mt_de-en	llama32-3b	reverse	+0.103 [+0.075, +0.131]	-0.181 [-0.206, -0.155]
mt_de-en	gemma3-4b	forward	-2.352 [-2.547, -2.168]	-0.002 [-0.012, +0.008]
mt_de-en	gemma3-4b	reverse	-1.008 [-1.122, -0.893]	-0.419 [-0.459, -0.378]
units	gemma3-12b	forward	-2.587 [-2.829, -2.358]	+0.016 [-0.005, +0.053]
units	gemma3-12b	reverse	-2.612 [-2.832, -2.405]	-0.381 [-0.503, -0.273]

Table 26: Gold-answer negative log-likelihood per token on 128 pairs (seed 17). Differences have descriptive 95% bootstrap intervals; negative values favor the first system.

Task	Model	SFT	Replay	mix5
fmt	llama32-3b	-0.018	+0.004	+0.010
mt_de-en	llama32-3b	-0.000	+0.048	+0.059
mt_de-en	gemma3-4b	+0.021	+0.012	+0.031
units	gemma3-12b	-0.013	+0.009	-0.026

Table 27: Forward/backward validation-gradient cosine on 16 pairs in LoRA coordinates. Inactive Gemma vision factors are excluded.

Task	Model	Direction	Base	SFT	mix5
fmt	llama32-3b	forward	0.695 / 0.719	1.000 / 1.000	1.000 / 1.000
fmt	llama32-3b	reverse	0.750 / 0.672	0.000 / 0.008	1.000 / 1.000
mt_de-en	llama32-3b	forward	0.344 / 0.195	0.484 / 0.234	0.445 / 0.242
mt_de-en	llama32-3b	reverse	0.062 / 0.180	0.000 / 0.000	0.320 / 0.562
mt_de-en	gemma3-4b	forward	0.375 / 0.289	0.461 / 0.398	0.359 / 0.273
mt_de-en	gemma3-4b	reverse	0.406 / 0.586	0.000 / 0.000	0.430 / 0.562
units	gemma3-12b	forward	0.828 / 0.875	1.000 / 1.000	0.992 / 1.000
units	gemma3-12b	reverse	0.586 / 0.547	1.000 / 0.977	1.000 / 1.000

Table 28: Fraction of 128 pairs where the gold answer outranks the copied input by total/per-token negative log-likelihood (seed 17).

Removed band	Matched control	Direction	Success difference
band0_remove	band0_scale	forward	-0.043 [-0.062, -0.025]
band0_remove	band0_scale	reverse	-0.006 [-0.014, +0.000]
band0_remove	band0_random101	forward	-0.039 [-0.057, -0.021]
band0_remove	band0_random101	reverse	-0.014 [-0.023, -0.004]
band0_remove	band0_random202	forward	-0.037 [-0.055, -0.020]
band0_remove	band0_random202	reverse	+0.000 [+0.000, +0.000]
band1_remove	band1_scale	forward	-0.010 [-0.021, +0.000]
band1_remove	band1_scale	reverse	+0.096 [+0.070, +0.121]
band1_remove	band1_random101	forward	-0.008 [-0.018, +0.000]
band1_remove	band1_random101	reverse	+0.094 [+0.068, +0.119]
band1_remove	band1_random202	forward	-0.008 [-0.018, +0.000]
band1_remove	band1_random202	reverse	+0.100 [+0.074, +0.125]
band2_remove	band2_scale	forward	-0.002 [-0.012, +0.006]
band2_remove	band2_scale	reverse	+0.084 [+0.061, +0.107]
band2_remove	band2_random101	forward	+0.000 [-0.010, +0.010]
band2_remove	band2_random101	reverse	+0.082 [+0.057, +0.107]
band2_remove	band2_random202	forward	+0.002 [-0.008, +0.012]
band2_remove	band2_random202	reverse	+0.088 [+0.062, +0.113]
band3_remove	band3_scale	forward	+0.002 [+0.000, +0.006]
band3_remove	band3_scale	reverse	+0.037 [+0.021, +0.055]
band3_remove	band3_random101	forward	+0.006 [+0.000, +0.014]
band3_remove	band3_random101	reverse	+0.033 [+0.018, +0.051]
band3_remove	band3_random202	forward	+0.008 [+0.002, +0.016]
band3_remove	band3_random202	reverse	+0.043 [+0.025, +0.061]

Table 29: Layer-removal effects relative to controls matched on retained norm: strict-success differences (percentage points), with descriptive 95% paired intervals; formatting/Llama3B, 512 pairs, seed 17.